Intelligent Security Posture Aware Service Function Chain Placement Using Deep Reinforcement Learning

Ali Raheel

CIIT/SP24-RCS-001/LHR

Supervisor: Dr. Muhammad Hasanain Chaudary

Department of Computer Science
COMSATS University Islamabad, Pakistan

Spring 2026

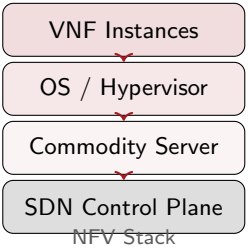
Outline

1. Introduction & Motivation
2. System Model
3. Security Posture Risk Model
4. Proposed Methodology
5. Experimental Results
6. Conclusions & Future Work

Background: Network Virtualization & NFV

Key Technologies

- **Software-Defined Networking (SDN)** decouples control from data plane
- **Network Function Virtualization (NFV)** replaces dedicated hardware with *Virtual Network Functions (VNFs)* on commodity servers
- Foundation of **5G/6G** network architectures (3GPP)
- Reduces CAPEX/OPEX; enables elastic scaling



The Challenge

Running VNFs on shared commodity hardware **expands the attack surface** and requires efficient, **security-aware resource orchestration**.

Service Function Chaining (SFC)

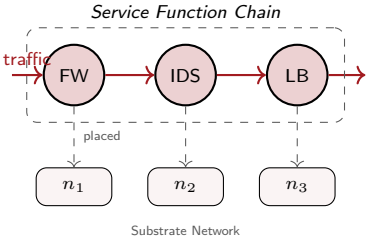
What is an SFC?

An **ordered sequence of VNFs** that network traffic must traverse to deliver an end-to-end service (e.g. Firewall → IDS → Load Balancer).

SFC Request Tuple

$$r = \langle \mathcal{V}_r,\ s_r^{\min},\ \ell_r^{\max},\ b_r^{\min},\ \tau_r,\ \delta_r \rangle$$

$\mathcal{V}_r$	ordered VNF sequence
$s_r^{\min}$	min. security score
$\ell_r^{\max}$	max. end-to-end latency
$b_r^{\min}$	min. bandwidth per hop
τ_r	time-to-live (TTL)
δ_r	hard isolation flag



The SFC Placement Problem

Multi-constraint Placement: assign each VNF v_k to substrate node n_k

Hard Constraints

- **Resource:** RAM, CPU, storage $\geq$ VNF demand
- **Latency:** $\sum L(n_k, n_{k+1}) \leq \ell_r^{\max}$
- **Bandwidth:** $B(n_k, n_{k+1}) \geq b_r^{\min}$
- **Security:** $s_{n_k}^{\text{eff}} \geq s_r^{\min}$
- **Isolation:** no node sharing if $\delta_r = 1$

Why Is It Hard?

- **NP-hard** in general (Santos et al. 2021)
- Dynamic, online arrival of requests
- Myopic decisions fragment the substrate
- Security posture changes over time
- Shared nodes create *blast-radius* risk

Existing Approaches

ILP (optimal but slow) · heuristics (fast but greedy) · meta-heuristics (expensive) — **none learn from the request stream**

Research Gaps & Objectives

Four Gaps in the Literature

1. Constraints treated as **soft penalties** — no hard feasibility guarantee
2. Security modelled **statically** — no dynamic posture, no incident history
3. Policies are **topology-blind** — flat feature vectors only
4. Evaluations use **different request streams** — unfair comparison

This Thesis Addresses Each Gap

1. **Action masking** ⇒ 100% feasibility guarantee
2. **Dynamic security posture risk model** with incident pressure & load
3. **GNN feature extractor** (GCN / GAT / GraphSAGE)
4. **Aligned RNG evaluation** ⇒ identical request streams

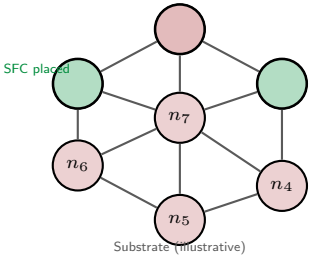
Substrate Network Model

Graph Model $G_S = (N, E)$

- **Node** n: resource vector $\mathbf{r}_n = (r_n^{\text{ram}}, r_n^{\text{cpu}}, r_n^{\text{stor}}, s_n)$
- **Link** (u, v): bandwidth b_{uv}, latency ℓ_{uv}
- Routing: shortest hop-count paths

Experimental Setup

Topology	Erdős-Rényi $G(25, 1.0)$
Node RAM	$[20, 120]$ GB
Node CPU	$[10, 60]$ cores
Storage	$[250, 1500]$ GB
Security score s_n	$[1, 5]$
Link BW	$[100, 500]$ Mbps
Link latency	$[5, 60]$ ms



TTL & Resource Release

Each SFC holds resources for τ_r steps; released via `tick()` — models dynamic, fragmenting NFV substrate.

MDP Formulation

$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$

- **State** $\mathbf{s}_t$: node feature matrix $\mathbf{F} \in \mathbb{R}^{25\times 20}$, global context $\mathbf{g} \in \mathbb{R}^{15}$, placement mask, node mask
- **Action**: select substrate node n_k for VNF v_k
- **Episode**: $T = 300$ SFC requests; $K_r = 3$ VNFs/chain $\Rightarrow$ up to 900 steps
- **Discount**: $\gamma = 0.99$

Objective

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{T} \gamma^t r_t \right]$$

Reward Structure

Intermediate (per VNF $k < K_r$):

$$r_t^{\text{step}} = 0.1 - 0.3\,\frac{L(n_{k-1}, n_k)}{\ell_r^{\max}}$$

Terminal acceptance:

$$r_T^{\text{acc}} = \big(1 - \lambda\rho_r - \mu\widehat{C}_r\big)\big(1 + \omega\overline{\mathsf{AR}}_w\big)$$

Terminal rejection:

$$r_T^{\text{rej}} = -1 \cdot \big(1 + \omega(1 - \overline{\mathsf{AR}}_w)\big)$$

$\lambda = 4,\ \mu = 0.05,\ \omega = 0.5,\ w = 10$

Dynamic Security Posture

Effective Security Score

Incident history reduces a node's security posture:

$$s_n^{\text{eff}}(t) = s_n \cdot \big(1 - \eta \cdot p_n(t)\big)$$

Incident pressure *decays geometrically* between events:

$$p_n(t+1) = 0.92 \cdot p_n(t)$$

Upon incident: $p_n \leftarrow p_n + 0.20$

Node Downtime

Realized incidents lock the node for $D_{\text{inc}} = 2$ ticks. Resources zeroed; no VNF may be placed.

Per-Node Incident Probability

$$q_n = q_0 \underbrace{\left(1 - \frac{s_n^{\text{eff}}}{s_{\max}}\right)^{1.6}}_{\text{security}} \underbrace{(1 + 0.7\,\ell_n)}_{\text{load}} \underbrace{(1 + p_n)}_{\text{pressure}}$$

$q_0 = 0.025$; q_n clamped to $[0, 1]$

Three Risk Drivers

1. **Security posture** (s_n^{eff}, p_n)
2. **Load** (ℓ_n = normalized VNF count)
3. **Incident pressure** (historical events)

Security Posture Risk Integral

Risk Integral $\rho_r \in [0, 1]$

$$\rho_r = \min\left(\frac{1}{|\Pi_r^{\text{dist}}|} \sum_{n \in \Pi_r^{\text{dist}}} q_n \cdot \text{SFC}(n) \cdot \frac{T_s}{\tau_{\max}},\ 1 \right)$$

- $\text{SFC}(n)$ = blast radius (co-located chains)
- $T_s = \min(\tau_r, 12)$ = exposure window
- Deterministic $\Rightarrow$ smooth gradient signal

Expected Lost Revenue $\widehat{C}_r$

$$\widehat{C}_r = \sum_n P_n^{\text{hit}} \cdot \text{SFC}(n) \cdot R_{\text{ttl}} \cdot D_{\text{inc}}$$

$$P_n^{\text{hit}} = 1 - (1 - q_n)^{T_s}$$

Stochastic Incident Simulation

Bernoulli trials *drive posture dynamics* (separate from reward):

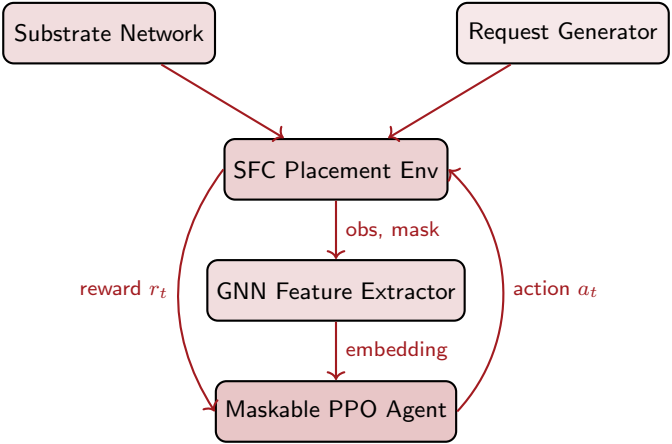
$$I_r = \sum_n \sum_{t=1}^{T_s} \mathbf{1}[\text{Bernoulli}(q_n) = 1]$$

Nodes hit $\Rightarrow$ pressure increment + downtime counter.

Design Insight

The **deterministic** ρ_r and $\widehat{C}_r$ are used in the *reward* for low-variance gradients; stochastic simulation is used only for *posture state updates*.

System Architecture



Maskable PPO + Action Masking

Why Maskable PPO?

- Infeasible actions receive logit $-\infty$ before softmax:

$$\pi(a|\mathbf{s}_t) = \text{softmax}(\mathbf{z}_t \odot \mathbf{m}_t^{\text{act}})$$

- **Zero probability** on constraint-violating nodes
- No need for per-constraint penalty terms
- Better sample efficiency: no wasted gradients

Mask Construction (per step)

1. Resource + security score check
2. Incident downtime check
3. Hard isolation check
4. Bandwidth & latency budget check
5. Safety fallback: always enable node 0

PPO Training Hyperparameters

Total timesteps	100,000
Rollout steps	4,096
Batch size	512
Update epochs	10
Learning rate	10^{-4}
Discount γ	0.99
GAE λ	0.97
Clip range ϵ	0.2
Entropy coeff.	0.01
Risk penalty λ	4.0
AR window w	10

GNN Feature Extractor

Node Feature Matrix $\mathbf{F} \in \mathbb{R}^{25\times 20}$

Indices	Group
0–3	Base resources + effective security
4	Avg. residual neighbor bandwidth
5	Distance from previous placement
6–8	Global resource shares
9–10	VNF density & SFC tenancy ratio
11–13	VNF fit ratios (RAM/CPU/storage)
14–19	Risk signals: p_n, ℓ_n, q_n, ELR

Global Context $\mathbf{g} \in \mathbb{R}^{15}$

VNF demands, SFC constraints, progress index, isolation flag, latency consumed, TTL, windowed AR

Three GNN Variants

- **GCN** — Graph Convolutional Network
- **GAT** — Graph Attention Network
- **GraphSAGE** — Inductive aggregation

Architecture

$$\mathbf{F} \xrightarrow{\text{GNN layers}} \text{node embeddings} \xrightarrow{\text{attn-pool}} \text{graph embedding} \xrightarrow{\oplus\,\mathbf{g}} \text{actor-critic heads}$$

Advantage over MLP

Captures topology-dependent patterns: node centrality, neighbourhood load, security posture clusters — invisible to flat feature vectors

Best-Fit Baseline

Best-Fit by Minimum Risk

At each VNF step: apply identical feasibility checks, then select:

$$n^* = \arg\min_{n \in \mathcal{V}_t^{\text{valid}}} \left(\rho_n^{\text{node}},\ w_n^{\text{waste}},\ n\right)$$

where the per-node risk score is:

$$\rho_n^{\text{node}} = \min\left(q_n \cdot \text{SFC}(n) \cdot \frac{T_s}{\tau_{\max}},\ 1\right)$$

Why a Strong Baseline?

Directly minimizes the **same** objective component as the DRL risk penalty. Tests whether long-horizon reasoning adds value.

Baseline Limitation

Myopic: greedy at each step. Cannot:

- Anticipate future demand patterns
- Preserve low-risk nodes for strict requests
- Trade short-term risk for long-term efficiency

Fair Evaluation

RNG state **aligned** before each algorithm run $\Rightarrow$ identical request sequences for unbiased comparison.

Experimental Setup

Configuration

Substrate	25-node complete graph
SFC requests	1,000 per evaluation
VNFs per chain	3 (fixed)
TTL	Truncated Exp.$[1, 25]$
Isolation prob.	10%
Min. security	$[1, 5]$ uniform
Hardware	Windows 10, Python 3.12

Evaluation Protocol

- Single long episode: 1,000 requests, no resets
- RNG state snapshotted & reset identically for both algorithms
- Best model checkpoint (by acceptance ratio) saved during training

Metrics Tracked

- Acceptance ratio
- Average risk integral ρ_r
- Substrate utilization
- Avg. SFCs / VNFs per node
- Average security margin
- Total & per-request revenue
- Total & per-request lost revenue
- Total realized incidents
- Average decision time (ms)

Comparative Results: Summary Table

Metric	Best-Fit	PPO	$\Delta\%$	Winner
Acceptance Ratio	0.9634	0.9550	-0.87	Best-Fit
Avg. Risk Integral ρ	0.0014	0.0006	-59.45	**PPO**
Avg. Security Margin	0.8525	1.7593	$+106.38$	**PPO**
Total Realized Incidents	35.20	8.20	-76.70	**PPO**
Total Lost Revenue	68.80	44.80	-34.88	**PPO**
Total Revenue	9319.80	9244.00	-0.81	Best-Fit
Avg. Revenue / Request	9.3198	9.2440	-0.81	Best-Fit
Avg. Lost Rev. / Request	0.0688	0.0448	-34.88	**PPO**

PPO Wins (Security)

76.7% fewer incidents · 59.5% lower risk · 2× security margin · 35% less lost revenue

Trade-off (Acceptance)

Best-Fit edges acceptance ratio by **0.87%** and total revenue by **0.81%** — a small price for large security gains.

Key Findings & Analysis

Action Masking Benefits

- **100% feasibility** of every accepted placement
- Improved sample efficiency — no wasted gradients on infeasible actions
- Simplified reward design: no per-constraint penalty terms needed

GNN vs. MLP Policies

- GNN captures **centrality, neighborhood load**, and **security posture clusters**
- Attention weights reveal which nodes the policy considers most important
- Structural awareness unavailable to flat feature vectors

PPO's Long-Horizon Advantage

- May accept *slightly riskier* early placements to **preserve low-risk nodes** for future strict requests
- Greedy Best-Fit cannot exhibit this strategy

Windowed AR Modifier

- Amplifies acceptance reward after high-AR phases
- Amplifies rejection penalty after low-AR phases
- Provides **curriculum-style** training signal
- Stabilizes training convergence

Thesis Contributions

1. **Maskable PPO placement agent** with hard-masked security posture constraints — guarantees feasibility by construction.
2. **Expected-loss security posture risk model** combining incident probability (q_n), blast radius ($\mathrm{SFC}(n)$), and TTL-proportional exposure.
3. **Topology-aware GNN feature extractor** (GCN / GAT / GraphSAGE) with attention aggregation and MLP global context fusion.
4. **Windowed acceptance ratio reward modifier** for curriculum-style, adaptive feedback during training.
5. **Fair comparative evaluation framework** with aligned RNG state ensuring identical request streams across algorithms.
6. **Comprehensive experimental analysis** across 11 metrics on 1,000 SFC requests.

Limitations & Future Work

Current Limitations

1. Fixed 25-node substrate; transfer to larger topologies not yet validated
2. 100K training steps may be insufficient for complex configurations
3. Simplified Bernoulli incident model (no temporal correlations or cascading failures)
4. Single baseline — no ILP or First-Fit comparison

Promising Directions

- **Larger/dynamic topologies** (100–500 nodes, GraphSAGE transfer)
- **Multi-agent placement** across administrative domains
- **Richer posture models** with threat intelligence feeds and cascade effects
- **Hierarchical DRL** (zone selection + VNF assignment)
- **Online/continual learning** with live posture telemetry
- **ETSI MANO integration** for real NFV testbed validation
- **Explainability** via GNN attention weight analysis

Conclusion

Summary

This thesis presents a **security posture aware Deep Reinforcement Learning framework** for SFC placement that:

- Guarantees **100% feasibility** of accepted placements via hard action masking
- Reduces **realized security incidents by 76.7%** and **risk integral by 59.5%** vs. Best-Fit
- Achieves **competitive acceptance ratio** ($< 1\%$ difference) with a much stronger security posture
- Introduces a **principled, deterministic risk model** linking node posture, load, and blast radius to reward
- Leverages **graph-structured topology awareness** through GNN message passing

Thank you!

Questions?

Appendix: Node Feature Details

Index	Group	Description
0–3	Base resources	Residual RAM, CPU, Storage (fraction of initial); Effective security $s_n^{\text{eff}}/s_{\max}$
4	Connectivity	Avg. residual bandwidth to neighbors / $b_{\max}$
5	Distance	Shortest-path latency from previous placement node / $\ell_{\max}$
6–8	Global resource shares	Fraction of all consumed RAM/CPU/Storage on this node
9	VNF global share	Fraction of all VNF instances hosted here
10	SFC tenancy ratio	$\mathrm{SFC}(n)/\max(\mathrm{VNF}(n),1)$: blast-radius proxy
11–13	VNF fit ratios	$d_v^r/\tilde{r}_n^r$ for RAM, CPU, Storage
14	Incident pressure	p_n
15	Load norm	$\mathrm{VNF}(n)/Q_{75}^{\mathrm{VNF}}$
16	Base incident prob.	q_n (Eq. in System Model)
17	Expected incidents	$q_n \cdot T_s/T_{\text{cap}}$
18	Node risk score	$= q_n$ (explicit risk slot)
19	Expected lost rev.	$1 - 1/(1 + P_n^{\text{hit}} \cdot \mathrm{SFC}(n) \cdot R_{\text{ttl}} \cdot D_{\text{inc}})$

Appendix: Global Context Vector

Index	Description
0–2	Current VNF demands: RAM/$d_{\max}^{\text{ram}}$, CPU/$d_{\max}^{\text{cpu}}$, Storage/$d_{\max}^{\text{stor}}$
3	SFC min. security score $s_r^{\min}/s_{\max}$
4	SFC max. latency $\ell_r^{\max}/\ell_{\max}$
5	SFC min. bandwidth $b_r^{\min}/b_{\max}$
6	VNF progress: current VNF index / $K_{\max}$
7	Hard isolation flag (1.0 if $\delta_r = 1$)
8	Latency progress: accumulated latency / $\ell_r^{\max}$
9	Normalized TTL: $(\tau_r - \tau_{\min})/(\tau_{\max} - \tau_{\min})$
10	Remaining VNF count / $K_{\max}$
11–13	Aggregate remaining VNF demands (RAM, CPU, Storage) normalized
14	Windowed acceptance ratio $\overline{\mathrm{AR}}_w$